

Chapter 6

Autoregressive Models and Transformers

Enhanced Exam Study Guide

Formula Formatting Fixed + Missing Lecture Details Added

What this updated version fixes

- All equations are rewritten using proper mathematical notation, displayed fractions, products, summations, masks, gates, norms, and matrix shapes.
- Missing information from the annotated lecture is inserted into the matching sections: RNN/LSTM parameter shapes, BPTT chain rule, attention diagram intuition, value-vector meaning, LayerNorm, byte-level BPE, the GPT training loop, top- k /temperature generation, and worked masked-attention weights.
- The same Chapter 2-style flow is preserved: exam map, equation sheet, concept sections, worked examples, sample questions, flashcards, and final checklist.

Sources used: Chapter 6 exam study guide and Chapter 6 annotated lecture.

Main exam goal: explain autoregressive modeling, RNN/LSTM limitations, self-attention, causal masking, BPE tokenization, and GPT-style decoder-only Transformers.

Contents

1 Quick Exam Map	3
1.1 Summary-to-Lecture Matching Map	3
2 Core Equation Sheet	4
3 Autoregressive Modeling	5
3.1 Binary 1-Step Markov Example	5
4 Training with Teacher Forcing vs. Sampling	5
5 RNNs: Hidden State as Compressed Memory	6
5.1 Learned Parameters and Shapes	6
5.2 Why RNNs Struggle	6
6 LSTMs: Gated Long-Term Memory	6
6.1 Gate Interpretation	7
7 BPTT and Gradient Difficulties	7
8 What a Transformer Computes	7
9 Self-Attention: Query, Key, and Value	8
9.1 Why Values Matter	8
9.2 Small Softmax Example	9
10 Causal Masking in Decoder-Only Models	9
10.1 Worked Masked-Attention Example	9
11 Multi-Head Attention	10
12 Transformer Block: MHA + FFN + Residuals + LayerNorm	10
12.1 LayerNorm Formula	10
12.2 FFN/MLP	10
13 Positional Information and BPE Tokenization	11
13.1 Byte-Pair Encoding	11
14 GPT-Style Decoder-Only Training and Generation	11
14.1 Generation Loop	12
15 Transformer Architecture Families	12
16 Sample Exam Questions with Strong Answers	12
17 Flashcards - All 80 Unique Questions	14

1 Quick Exam Map

Exam focus

By the end of this chapter, you should be able to write and explain:

- the autoregressive factorization $p(x_{1:T}) = \prod_{t=1}^T p(x_t | x_{<t})$;
- the difference between teacher forcing and sampling;
- why RNNs/LSTMs are sequential and why BPTT causes vanishing/exploding gradients;
- the LSTM gates i_t, f_t, o_t and the memory update c_t ;
- how self-attention builds Q, K, V , scores, softmax weights, and weighted sums of values;
- why decoder-only Transformers need a causal mask;
- how multi-head attention, LayerNorm, residuals, FFNs, positional embeddings, BPE, and the LM head fit together.

1.1 Summary-to-Lecture Matching Map

Guide section	Original focus	Added missing details in this version
Autoregressive models	Sequence probability and next-token prediction	Binary 1-step Markov example, Bernoulli likelihood, teacher-forcing loss, sampling loop.
RNNs	Hidden state as memory	Learnable parameters and shapes: embeddings, input-to-hidden, hidden-to-hidden, output projection.
LSTMs	Gated memory	Gate parameter shapes, gate roles, cell-state overwrite/-copy interpretation.
BPTT	Unrolled gradients	Chain-rule path and why Jacobian products lead to vanishing/exploding gradients.
Transformers	Contextual vectors	Explanation of contextualization and why the word vector changes with context.
Attention	Q/K/V equations	The “too/car/elephant” intuition, value-vector meaning, numeric softmax example.
Causal masking	Prevent future leakage	Mask matrix, masked scores, and worked attention weights.
Transformer block	MHA + FFN + residuals + LN	LayerNorm equation, GELU-MLP role, pre-LN stability.
Tokenization/GPT	BPE and decoder-only objective	Byte-level BPE algorithm, Tiny Shakespeare data loop, block-size batching, top- k /temperature sampling.

2 Core Equation Sheet

Formula

Autoregressive factorization

$$p(x_{1:T}) = \prod_{t=1}^T p(x_t \mid x_{1:t-1}) = \prod_{t=1}^T p(x_t \mid x_{<t}).$$

Categorical next-token model

$$p(s_t \mid s_{<t}) = \text{Categorical}(\pi_t), \quad \pi_t = \text{softmax}(z_t),$$

$$\Pr(s_t = i \mid s_{<t}) = \pi_{t,i}, \quad i = 1, \dots, V.$$

Binary AR model

$$\hat{p}_t = p_\theta(x_t = 1 \mid x_{t-1}) = \sigma(wx_{t-1} + b),$$

$$p_\theta(x_t \mid x_{t-1}) = \hat{p}_t^{x_t} (1 - \hat{p}_t)^{1-x_t}, \quad x_t \in \{0, 1\}.$$

RNN

$$e(x_t) = E[x_t] \in \mathbb{R}^d,$$

$$h_t = \phi(W_{xh}e(x_t) + W_{hh}h_{t-1} + b_h),$$

$$z_{t+1} = W_{yh}h_t + b_y, \quad p(x_{t+1} \mid x_{\leq t}) = \text{softmax}(z_{t+1}).$$

LSTM gates

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i),$$

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f),$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o),$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c),$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t,$$

$$h_t = o_t \odot \tanh(c_t).$$

Self-attention

$$Q = XW_Q,$$

$$K = XW_K,$$

$$V = XW_V,$$

$$S = \frac{QK^\top}{\sqrt{d_k}},$$

$$A = \text{softmax}(S + M),$$

$$Y = AV.$$

Causal mask

$$M_{t,u} = \begin{cases} 0, & u \leq t, \\ -\infty, & u > t. \end{cases}$$

Multi-head attention and Transformer block

$$\text{head}_h = \text{Attention}(XW_Q^{(h)}, XW_K^{(h)}, XW_V^{(h)}),$$

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O,$$

$$X' = X + \text{MHA}(\text{LN}(X)),$$

$$X_{\text{next}} = X' + \text{FFN}(\text{LN}(X')),$$

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2.$$

3 Autoregressive Modeling

An autoregressive model defines a joint probability over a sequence by decomposing it into next-step conditionals. This is the central probabilistic idea behind language models.

$$x_{1:T} = (x_1, x_2, \dots, x_T), \quad p(x_{1:T}) = \prod_{t=1}^T p(x_t \mid x_{<t}).$$

For language modeling, each token s_t belongs to a finite vocabulary $\{1, \dots, V\}$. The model produces logits $z_t \in \mathbb{R}^V$ and probabilities

$$\pi_t = \text{softmax}(z_t), \quad p(s_t \mid s_{<t}) = \text{Categorical}(\pi_t).$$

Added from the annotated lecture

The annotated lecture emphasizes that this same factorization explains two different stages: during **training**, the model conditions on the true previous tokens; during **sampling**, it conditions on tokens it generated itself. This is why a model can be trained with next-token prediction but used to generate a whole sequence.

3.1 Binary 1-Step Markov Example

For a binary alternating pattern, $x_t \in \{0, 1\}$ and the ideal rule is

$$x_t = 1 - x_{t-1}, \quad t \geq 2.$$

A simple 1-step model assumes

$$p_\theta(x_t \mid x_{1:t-1}) = p_\theta(x_t \mid x_{t-1}).$$

It predicts the Bernoulli probability

$$\hat{p}_t = p_\theta(x_t = 1 \mid x_{t-1}) = \sigma(wx_{t-1} + b).$$

The likelihood for the observed binary target is

$$p_\theta(x_t \mid x_{t-1}) = \hat{p}_t^{x_t} (1 - \hat{p}_t)^{1-x_t}.$$

The negative log-likelihood becomes binary cross-entropy:

$$\mathcal{L}(\theta) = - \sum_{t=2}^T [x_t \log \hat{p}_t + (1 - x_t) \log(1 - \hat{p}_t)].$$

4 Training with Teacher Forcing vs. Sampling

Teacher forcing means that when predicting a target token, the model receives the true prefix from the dataset.

$$\mathcal{L}(\theta) = - \sum_{t=1}^{T-1} \log p_\theta(x_{t+1} \mid x_{\leq t}).$$

At generation time, the next token is sampled or selected from the model distribution:

$$x_{t+1} \sim \text{Categorical}(p_\theta(\cdot \mid x_{\leq t})).$$

Exam focus

The exam point is the mismatch: training uses ground-truth prefixes, but sampling uses model-generated prefixes. This mismatch is called **exposure bias**. If the model makes an early mistake, later predictions are conditioned on a prefix it may not have seen during training.

5 RNNs: Hidden State as Compressed Memory

An RNN processes tokens sequentially. The hidden state h_t is a compressed summary of the past.

$$h_t = \phi(W_{xh}e(x_t) + W_{hh}h_{t-1} + b_h),$$

$$p(x_{t+1} \mid x_{\leq t}) = \text{softmax}(W_{yh}h_t + b_y).$$

5.1 Learned Parameters and Shapes

Assume vocabulary size V , embedding dimension d , and hidden size H .

$$\begin{array}{lll} E \in \mathbb{R}^{V \times d}, & W_{xh} \in \mathbb{R}^{H \times d}, & W_{hh} \in \mathbb{R}^{H \times H}, \\ b_h \in \mathbb{R}^H, & W_{yh} \in \mathbb{R}^{V \times H}, & b_y \in \mathbb{R}^V. \end{array}$$

The full parameter set is

$$\theta = \{E, W_{xh}, W_{hh}, b_h, W_{yh}, b_y\}.$$

Added from the annotated lecture

The lecture notes stress that there is one neural network reused at every time step. The weights do not change from step to step; instead, the same parameters process each input token and update the hidden state.

5.2 Why RNNs Struggle

The bottleneck is structural: all relevant history must be compressed into one vector h_t . Also, computation is sequential because h_t cannot be computed before h_{t-1} :

$$h_1 \rightarrow h_2 \rightarrow \dots \rightarrow h_T.$$

This limits parallelism and makes long-range dependencies difficult.

6 LSTMs: Gated Long-Term Memory

An LSTM adds a memory cell c_t and three gates. The gates are learned soft controls in $(0, 1)$.

$$\begin{aligned} i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i), \\ f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f), \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o), \\ \tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c), \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \\ h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

6.1 Gate Interpretation

- $f_t \odot c_{t-1}$: how much old memory is kept.
- $i_t \odot \tilde{c}_t$: how much new candidate memory is written.
- $o_t \odot \tanh(c_t)$: how much memory is exposed as the hidden output.

$$\begin{aligned} f_t \approx 1, i_t \approx 0 &\implies c_t \approx c_{t-1}. \\ f_t \approx 0, i_t \approx 1 &\implies c_t \approx \tilde{c}_t. \end{aligned}$$

Added from the annotated lecture

The annotated lecture explains that U_i , U_f , and U_o are context weights: they let the previous hidden state h_{t-1} influence whether the model writes, forgets, or reveals memory. Thus the gates are not only input-driven; they are also context-aware.

7 BPTT and Gradient Difficulties

For an RNN/LSTM,

$$h_t = f_\theta(h_{t-1}, x_t), \quad t = 1, \dots, T.$$

For next-token prediction,

$$\mathcal{L}(\theta) = \sum_{t=1}^{T-1} \ell_t(\theta), \quad \ell_t(\theta) = -\log p_\theta(x_{t+1} \mid x_{\leq t}).$$

BPTT applies the chain rule through the unrolled recurrence:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_{t=1}^{T-1} \frac{\partial \ell_t}{\partial \theta}.$$

A term may contain a long product of Jacobians:

$$\frac{\partial \ell_t}{\partial \theta} = \sum_{k=1}^t \frac{\partial \ell_t}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \dots \frac{\partial h_k}{\partial \theta}.$$

If repeated Jacobians shrink norms, gradients vanish. If they amplify norms, gradients explode.

8 What a Transformer Computes

A Transformer computes a contextualized vector for every token position. Start with tokens $x_1, \dots, x_T$. After token and position embeddings,

$$h_t^{(0)} = W_e[x_t] + W_p[t].$$

Each layer updates a token by mixing information from allowed positions:

$$h_t^{(\ell+1)} = \text{Block}\left(h_1^{(\ell)}, \dots, h_T^{(\ell)}\right)_t.$$

After L layers,

$$h_t^{(L)} = f_\theta(x_1, \dots, x_T; t).$$

Exam focus

A token representation is contextual if it depends on the sentence around it. For example, the vector for “bank” should differ in “river bank” and “bank account” because the context changes the meaning.

9 Self-Attention: Query, Key, and Value

Let the input sequence matrix be

$$X \in \mathbb{R}^{T \times d}.$$

A single attention head computes

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

where

$$W_Q, W_K \in \mathbb{R}^{d \times d_k}, \quad W_V \in \mathbb{R}^{d \times d_v}.$$

The score from query token i to key token j is

$$s_{ij} = \frac{q_i k_j^\top}{\sqrt{d_k}}.$$

The attention weights are

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_u \exp(s_{iu})}, \quad \sum_j \alpha_{ij} = 1, \quad \alpha_{ij} \geq 0.$$

The output for token i is

$$y_i = \sum_j \alpha_{ij} v_j.$$

In matrix form,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Added from the annotated lecture

The annotated lecture explains the “too” example: in the sentence “the pink elephant tried to get into the car but it was too ...”, the token “too” should attend strongly to words such as “car”, “elephant”, and “it”. Attention acts like a soft search-and-retrieve mechanism over the earlier sentence.

9.1 Why Values Matter

The key k_j is used for matching. The value v_j is the information payload that gets mixed into the output if token j is relevant.

$$v_j = x_j W_V \in \mathbb{R}^{d_v}.$$

A value vector can encode multiple learned features, such as size, object type, syntactic role, or semantic relation. The model learns those features because they help reduce next-token loss.

9.2 Small Softmax Example

Suppose the query is from “too” and the three raw scores are

$$s_{\text{car}} = 2.0, \quad s_{\text{elephant}} = 1.0, \quad s_{\text{pink}} = 0.0.$$

Then

$$\begin{aligned}\alpha_{\text{car}} &= \frac{e^2}{e^2 + e^1 + e^0} \approx 0.665, \\ \alpha_{\text{elephant}} &\approx 0.245, \\ \alpha_{\text{pink}} &\approx 0.090.\end{aligned}$$

The contextual output is

$$y \approx 0.665v_{\text{car}} + 0.245v_{\text{elephant}} + 0.090v_{\text{pink}}.$$

10 Causal Masking in Decoder-Only Models

In GPT-style self-attention, token t may attend only to positions $u \leq t$.

$$M_{t,u} = \begin{cases} 0, & u \leq t, \\ -\infty, & u > t. \end{cases}$$

The masked attention weights are

$$\alpha_{t,u} = \text{softmax}_u \left(\frac{q_t k_u^\top}{\sqrt{d_k}} + M_{t,u} \right).$$

Future positions receive zero probability because

$$\exp(-\infty) = 0.$$

10.1 Worked Masked-Attention Example

For $T = 4$, a causal mask has allowed positions below and on the diagonal:

$$\text{allowed} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}.$$

A possible masked-score matrix from the lecture example is

$$S + M = \begin{bmatrix} -2.857 & -\infty & -\infty & -\infty \\ 0.164 & 2.260 & -\infty & -\infty \\ 0.234 & 0.585 & -0.854 & -\infty \\ -3.177 & -0.115 & -3.591 & 2.216 \end{bmatrix}.$$

Applying row-wise softmax gives attention weights

$$A \approx \begin{bmatrix} 1.000 & 0 & 0 & 0 \\ 0.109 & 0.891 & 0 & 0 \\ 0.363 & 0.515 & 0.122 & 0 \\ 0.004 & 0.088 & 0.003 & 0.905 \end{bmatrix}.$$

The final attention output is

$$Y = AV.$$

11 Multi-Head Attention

Instead of one attention mechanism, a Transformer uses H heads. If the model dimension is d and heads are evenly split, a typical choice is

$$d_k = d_v = \frac{d}{H}.$$

For head h ,

$$\begin{aligned} Q^{(h)} &= XW_Q^{(h)}, & K^{(h)} &= XW_K^{(h)}, & V^{(h)} &= XW_V^{(h)}, \\ Y^{(h)} &= \text{softmax}\left(\frac{Q^{(h)}(K^{(h)})^\top}{\sqrt{d_k}} + M\right) V^{(h)}. \end{aligned}$$

Then

$$\text{MHA}(X) = \text{Concat}(Y^{(1)}, \dots, Y^{(H)})W_O.$$

12 Transformer Block: MHA + FFN + Residuals + LayerNorm

A common pre-LayerNorm block is

$$\begin{aligned} X' &= X + \text{MHA}(\text{LN}(X)), \\ X_{\text{next}} &= X' + \text{FFN}(\text{LN}(X')). \end{aligned}$$

12.1 LayerNorm Formula

For a token vector $x_t \in \mathbb{R}^d$, LayerNorm computes

$$\begin{aligned} \mu_t &= \frac{1}{d} \sum_{i=1}^d x_{t,i}, \\ \sigma_t^2 &= \frac{1}{d} \sum_{i=1}^d (x_{t,i} - \mu_t)^2, \\ \hat{x}_{t,i} &= \frac{x_{t,i} - \mu_t}{\sqrt{\sigma_t^2 + \epsilon}}, \\ \text{LN}(x_t)_i &= \gamma_i \hat{x}_{t,i} + \beta_i, \quad \gamma, \beta \in \mathbb{R}^d. \end{aligned}$$

Added from the annotated lecture

LayerNorm is applied independently for each token, not across the batch. This makes it suitable for autoregressive generation, where the model may generate one sequence or one token at a time.

12.2 FFN/MLP

The FFN is applied independently to each token:

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2.$$

In GPT-style models, σ is often GELU:

$$\text{GELU}(x) \approx \frac{1}{2}x \left[1 + \tanh\left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)\right) \right].$$

A good mental model is:

- attention decides *what information to read from other tokens*;
- the MLP decides *how to transform the mixed information into useful features*.

13 Positional Information and BPE Tokenization

Self-attention by itself is permutation-invariant. Learned positional embeddings make the input order explicit:

$$x_t^{(0)} = W_e[s_t] + W_p[t].$$

13.1 Byte-Pair Encoding

Language models use tokens from a finite vocabulary rather than raw words. BPE builds subword tokens by repeated merging.

Let a sequence of base symbols be

$$w = (c_1, c_2, \dots, c_n).$$

BPE repeatedly counts adjacent pairs

$$(c_1, c_2), (c_2, c_3), \dots, (c_{n-1}, c_n),$$

selects the most frequent pair,

$$(a, b) = \arg \max_{(u, v)} \text{count}(u, v),$$

and merges it into a new symbol ab . This repeats until the target vocabulary size is reached.

Added from the annotated lecture

Byte-level BPE starts from the 256 possible UTF-8 byte values. This guarantees that every Unicode string can be represented, so the model avoids unknown characters while still learning compact frequent substrings.

14 GPT-Style Decoder-Only Training and Generation

A GPT-style model is a decoder-only Transformer trained to predict the next token. Given a tokenized sequence

$$s_1, s_2, \dots, s_T,$$

training blocks are usually shifted by one:

$$x = [s_1, s_2, \dots, s_{T-1}], \quad y = [s_2, s_3, \dots, s_T].$$

The Transformer produces hidden states

$$H = \text{Transformer}(x) \in \mathbb{R}^{(T-1) \times d},$$

and vocabulary logits

$$Z = HW_{\text{out}} + b_{\text{out}} \in \mathbb{R}^{(T-1) \times V}.$$

The per-position cross-entropy is

$$\ell_t = -\log p_\theta(y_t \mid x_{\leq t}) = -Z_{t,y_t} + \log \left(\sum_{v=1}^V \exp(Z_{t,v}) \right).$$

The total loss is

$$\mathcal{L}(\theta) = \sum_{t=1}^{T-1} \ell_t = - \sum_{t=1}^{T-1} \log p_{\theta}(y_t \mid x_{\leq t}).$$

14.1 Generation Loop

At each step:

1. keep the last B tokens, where B is the block size or context window;
2. compute logits for the last position;
3. optionally apply temperature τ :

$$z'_v = \frac{z_v}{\tau};$$

4. optionally apply top- k filtering by keeping only the k largest logits;
5. sample

$$x_{t+1} \sim \text{Categorical}(\text{softmax}(z'));$$

6. append x_{t+1} and repeat.

15 Transformer Architecture Families

Family	Attention pattern	Typical use
Encoder-only	Bidirectional: token t can attend to all positions.	Understanding tasks such as classification, tagging, retrieval, and sentence embedding.
Decoder-only	Causal: token t attends only to positions $u \leq t$.	Autoregressive generation, text completion, dialogue, and code generation.
Encoder-decoder	Encoder reads the source bidirectionally; decoder generates target autoregressively with cross-attention to encoder states.	Sequence-to-sequence tasks such as translation and summarization.

16 Sample Exam Questions with Strong Answers

Question 1

Why is the autoregressive factorization both a probabilistic statement and a training strategy?

Answer. It is probabilistic because it assigns a likelihood to a full sequence by multiplying valid conditional distributions:

$$p(x_{1:T}) = \prod_{t=1}^T p(x_t \mid x_{<t}).$$

It is a training strategy because maximizing the sequence likelihood becomes next-token prediction at every position. Cross-entropy directly trains the model to improve these conditionals.

Question 2

Why are separate query, key, and value projections needed in self-attention?

Answer. Queries represent what the current token is looking for, keys represent what each token offers for relevance matching, and values represent the content retrieved after relevance is decided. This separation lets the model learn one geometry for matching and another representation for information transfer.

Question 3

Why is causal masking essential in a decoder-only model?

Answer. A decoder-only model must learn $p(x_t \mid x_{<t})$. If future tokens were visible, the model could inspect the answer during training. The causal mask forces

$$M_{t,u} = \begin{cases} 0, & u \leq t, \\ -\infty, & u > t, \end{cases}$$

so future attention weights become zero after softmax.

Question 4

How do Transformers address the two main limitations of LSTMs?

Answer. Transformers remove recurrence during training by processing all positions in parallel with self-attention. They also create short paths between distant tokens: one token can attend directly to another in a single layer, instead of passing information through many recurrent states.

Question 5

Why is tokenization not a minor preprocessing detail?

Answer. Tokenization determines vocabulary size, sequence length, and the granularity of learned patterns. Byte-level BPE avoids unknown characters while producing shorter sequences than character-level modeling and more flexibility than word-level tokenization.

Question 6

What is the difference between attention and the FFN inside a Transformer block?

Answer. Attention mixes information across positions:

$$Y = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V.$$

The FFN transforms each token independently:

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2.$$

Together, they allow context mixing and nonlinear feature transformation.

17 Flashcards - All 80 Unique Questions

The following flashcards preserve the full review set, but formulas are rewritten using mathematical notation.

Flashcard 1

Q: What is the core definition of an autoregressive (AR) model in sequence modeling?

A: It defines a probability distribution over a sequence by factorizing it into a product of next-step conditionals.

Flashcard 2

Q: Write the autoregressive factorization for a sequence $x_{1:T}$.

A: $p(x_{1:T}) = \prod_{t=1}^T p(x_t \mid x_{1:t-1}) = \prod_{t=1}^T p(x_t \mid x_{<t})$.

Flashcard 3

Q: In language modeling, what does $\text{Categorical}(\pi_t)$ represent?

A: It is the conditional distribution over the next token, where $\pi_t = \text{softmax}(z_t)$.

Flashcard 4

Q: What occurs during teacher forcing in autoregressive training?

A: The model conditions on the ground-truth prefix $x_{<t}$ to predict the next token x_t .

Flashcard 5

Q: How does sampling differ from training in autoregressive generation?

A: During sampling, the model conditions on its own previously generated tokens rather than ground-truth tokens.

Flashcard 6

Q: In a 1-step Markov AR model for binary digits, what determines $p(x_t)$?

A: Only the most recent digit x_{t-1} determines the next-step probability.

Flashcard 7

Q: What is the purpose of $\sigma(wx_{t-1} + b)$ in a binary AR model?

A: It gives $p_\theta(x_t = 1 \mid x_{t-1})$, a probability between 0 and 1.

Flashcard 8

Q: Write the Bernoulli PMF for a binary target $x \in \{0, 1\}$.

A: $P(X = x) = p^x(1 - p)^{1-x}$.

Flashcard 9

Q: What is the primary role of the hidden state h_t in an RNN?

A: It maintains a compressed summary or memory of previous tokens in the sequence.

Flashcard 10

Q: What is the main computational bottleneck of RNNs?

A: RNNs are sequential: h_t depends on h_{t-1} , preventing full parallelization over time.

Flashcard 11

Q: What training algorithm computes gradients in RNNs by unrolling through time?

A: Backpropagation Through Time (BPTT).

Flashcard 12

Q: What mathematical problem often occurs during BPTT across many steps?

A: Vanishing or exploding gradients.

Flashcard 13

Q: How does an LSTM address the weakness of vanilla RNNs?

A: It introduces a memory cell and gates that learn when to forget, write, and expose information.

Flashcard 14

Q: What is the LSTM cell state c_t designed to do?

A: It carries long-term information through time, acting like a memory highway.

Flashcard 15

Q: Name the three main LSTM gates.

A: Input gate i_t , forget gate f_t , and output gate o_t .

Flashcard 16

Q: What does the forget gate f_t control?

A: It controls how much of the old memory c_{t-1} is kept or erased.

Flashcard 17

Q: What does the input gate i_t control?

A: It controls how much new candidate information $\tilde{c}_t$ is written into memory.

Flashcard 18

Q: What does the output gate o_t control?

A: It controls how much of the memory is exposed through the hidden state h_t .

Flashcard 19

Q: Write the LSTM memory update equation.

A: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$.

Flashcard 20

Q: Write the LSTM hidden-state equation.

A: $h_t = o_t \odot \tanh(c_t)$.

Flashcard 21

Q: What does $f_t \approx 1$ and $i_t \approx 0$ imply?

A: The LSTM copies old memory forward almost unchanged.

Flashcard 22

Q: What does $f_t \approx 0$ and $i_t \approx 1$ imply?

A: The LSTM overwrites old memory with new candidate content.

Flashcard 23

Q: Why do LSTMs still have a scalability limitation?

A: They still process tokens sequentially, so parallelism over time remains limited.

Flashcard 24

Q: What is exposure bias?

A: The mismatch between training on ground-truth prefixes and sampling from model-generated prefixes.

Flashcard 25

Q: What is a Transformer's primary internal role?

A: It computes contextualized representations for every token position.

Flashcard 26

Q: What does contextualized representation mean?

A: A token vector depends on the surrounding allowed context, not just the token identity.

Flashcard 27

Q: How does GPT-style attention differ from BERT-style attention?

A: GPT uses causal attention to past tokens only; BERT uses bidirectional attention to all tokens.

Flashcard 28

Q: What is self-attention replacing from RNN/LSTM models?

A: It replaces recurrence as the main way tokens exchange contextual information.

Flashcard 29

Q: What are Q , K , and V in self-attention?

A: Queries, keys, and values computed from learned linear projections of token states.

Flashcard 30

Q: What is the role of a query q_i ?

A: It represents what token i is looking for.

Flashcard 31

Q: What is the role of a key k_j ?

A: It represents what token j offers for relevance matching.

Flashcard 32

Q: What is the role of a value v_j ?

A: It contains the content retrieved from token j after attention weights are computed.

Flashcard 33

Q: Write the scaled dot-product attention score between tokens i and j .

A: $s_{ij} = \frac{q_i k_j^\top}{\sqrt{d_k}}$.

Flashcard 34

Q: Why do we divide $QK^\top$ by $\sqrt{d_k}$?

A: To prevent attention scores from becoming too large as the key/query dimension grows, stabilizing softmax.

Flashcard 35

Q: How are attention weights α_{ij} computed?

A: $\alpha_{ij} = \text{softmax}_j(s_{ij})$, optionally after adding a mask M .

Flashcard 36

Q: Write the attention output for token i .

A: $\text{out}_i = \sum_j \alpha_{ij} v_j$.

Flashcard 37

Q: Write matrix-form scaled dot-product attention.

A: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$.

Flashcard 38

Q: What is the causal mask formula for decoder-only attention?

A: $M_{t,u} = 0$ if $u \leq t$, and $M_{t,u} = -\infty$ if $u > t$.

Flashcard 39

Q: Why does $-\infty$ in the mask block future tokens?

A: After softmax, $\exp(-\infty) = 0$, so masked future positions receive zero attention weight.

Flashcard 40

Q: What would happen if causal masking were removed during training?

A: The model could see future tokens and would not learn a valid $p(x_t | x_{<t})$.

Flashcard 41

Q: What is multi-head attention?

A: Several attention heads run in parallel with different learned projections, then their outputs are concatenated and projected.

Flashcard 42

Q: Write the multi-head attention combination formula.

A: $\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O$.

Flashcard 43

Q: Why is multi-head attention more expressive than one attention head?

A: Different heads can specialize in different relationships and represent several context patterns at once.

Flashcard 44

Q: Why do Transformers need positional information?

A: Self-attention is permutation-invariant without position, so the model needs positions to understand order.

Flashcard 45

Q: Write the usual initial token representation with learned position embeddings.

A: $x_t^{(0)} = W_e[s_t] + W_p[t]$.

Flashcard 46

Q: What is a residual connection in a Transformer block?

A: It adds the input of a sublayer back to its output, helping preserve information and gradients.

Flashcard 47

Q: Why is LayerNorm used in Transformer blocks?

A: It stabilizes activations and optimization by normalizing features within each token representation.

Flashcard 48

Q: What does the FFN/MLP do inside a Transformer block?

A: It applies a nonlinear transformation independently to each token after attention has mixed context.

Flashcard 49

Q: Write a standard FFN equation.

A: $\text{FFN}(x) = W_2\sigma(W_1x + b_1) + b_2$, often with GELU as σ .

Flashcard 50

Q: Write the common pre-LN Transformer block update.

A: $X' = X + \text{MHA}(\text{LN}(X))$, $X_{\text{next}} = X' + \text{FFN}(\text{LN}(X'))$.

Flashcard 51

Q: What is the path length advantage of attention over LSTM for long-range dependencies?

A: Self-attention can connect two positions in $O(1)$ layer path length, while RNN/LSTM paths grow with distance.

Flashcard 52

Q: What is the time/memory complexity of standard self-attention in sequence length T ?

A: $O(T^2)$, because each token compares with every token.

Flashcard 53

Q: Why can LSTMs still be useful in some settings?

A: They can be more memory-efficient for streaming or extremely long sequences when quadratic attention is too expensive.

Flashcard 54

Q: What is Byte-Pair Encoding (BPE)?

A: A subword tokenization method that repeatedly merges frequent adjacent symbols into larger tokens.

Flashcard 55

Q: Why is byte-level BPE useful?

A: Starting from UTF-8 bytes allows the tokenizer to represent any text without unknown characters.

Flashcard 56

Q: What is the main tradeoff BPE solves?

A: It is more compact than character-level modeling while more flexible than fixed word-level tokenization.

Flashcard 57

Q: What is a token vocabulary V ?

A: The finite set of token IDs the language model can output probabilities over.

Flashcard 58

Q: How does tokenization affect training efficiency?

A: It changes sequence length, vocabulary size, and the granularity of learned patterns.

Flashcard 59

Q: What is a decoder-only Transformer?

A: A Transformer stack using causal self-attention, trained to predict next tokens autoregressively.

Flashcard 60

Q: What is the GPT-style training objective?

A: Minimize cross-entropy: $\mathcal{L} = -\sum_t \log p_\theta(x_{t+1} \mid x_{\leq t})$.

Flashcard 61

Q: What are logits in a language model?

A: Unnormalized scores over the vocabulary before softmax.

Flashcard 62

Q: How are logits converted to token probabilities?

A: $\pi_t = \text{softmax}(z_t)$.

Flashcard 63

Q: What is the generation loop for a GPT-style model?

A: Given a prompt, compute next-token probabilities, choose or sample a token, append it, and repeat.

Flashcard 64

Q: Why is Transformer training more parallel than RNN training?

A: All positions can be processed in one forward pass with a causal mask, instead of computing hidden states step by step.

Flashcard 65

Q: Why is autoregressive generation still sequential?

A: The model must generate token $t + 1$ before it can use that token to generate later tokens.

Flashcard 66

Q: What is a context window or block size?

A: The maximum number of previous tokens the model can attend to at once.

Flashcard 67

Q: What does temperature control during sampling?

A: It rescales logits to make the output distribution sharper or flatter.

Flashcard 68

Q: What is greedy decoding?

A: Choosing $\arg \max$ token at each step instead of sampling from the distribution.

Flashcard 69

Q: What is top- k sampling?

A: Sampling only from the k most probable next tokens after renormalization.

Flashcard 70

Q: What is top- p /nucleus sampling?

A: Sampling from the smallest set of tokens whose cumulative probability exceeds p .

Flashcard 71

Q: What is an encoder-only Transformer best suited for?

A: Understanding tasks such as classification, tagging, and retrieval where bidirectional context is useful.

Flashcard 72

Q: What is a decoder-only Transformer best suited for?

A: Autoregressive generation such as text completion and dialogue.

Flashcard 73

Q: What is an encoder-decoder Transformer best suited for?

A: Sequence-to-sequence tasks such as translation and summarization.

Flashcard 74

Q: What does attention as content-based retrieval mean?

A: Tokens use queries to retrieve relevant information from other tokens using learned keys and values.

Flashcard 75

Q: Why is the output vector for 'bank' different in 'river bank' and 'bank account'?

A: Attention lets the representation incorporate context words such as 'river' or 'account'.

Flashcard 76

Q: What does the embedding matrix E learn in RNNs or Transformers?

A: It maps token IDs to dense vectors that capture useful syntactic and semantic properties.

Flashcard 77

Q: What is the difference between token embeddings and position embeddings?

A: Token embeddings encode token identity; position embeddings encode order/location.

Flashcard 78

Q: Why do residual connections help deep Transformers?

A: They create easier gradient paths and allow layers to refine rather than completely replace representations.

Flashcard 79

Q: What is the final language-model head?

A: A linear projection from final hidden states to vocabulary logits.

Flashcard 80

Q: Why is cross-entropy the natural loss for next-token prediction?

A: Because the next token is categorical and cross-entropy equals negative log-likelihood.

18 One-Page Final Review Checklist

- Can you write and explain $p(x_{1:T}) = \prod_t p(x_t | x_{<t})$?
- Can you explain teacher forcing, sampling, and exposure bias?
- Can you write the RNN update and explain the hidden-state bottleneck?
- Can you write the LSTM gate equations and interpret i_t , f_t , o_t , c_t , and h_t ?
- Can you explain BPTT as a chain-rule computation through time?
- Can you define Q , K , and V and write $S = QK^\top / \sqrt{d_k}$?
- Can you explain why softmax turns scores into weights?
- Can you explain why value vectors are the information payload?
- Can you write the causal mask and explain why $-\infty$ blocks future tokens?
- Can you compare single-head and multi-head attention?
- Can you write the pre-LN Transformer block equations?
- Can you explain why LayerNorm is used instead of BatchNorm in autoregressive generation?
- Can you describe byte-level BPE and why it avoids unknown characters?
- Can you explain the GPT-style shifted training pair $x = [s_1, \dots, s_{T-1}]$, $y = [s_2, \dots, s_T]$?
- Can you compare encoder-only, decoder-only, and encoder-decoder Transformers?

Exam focus

Most important chain to memorize:

tokens $\rightarrow$ embeddings + positions $\rightarrow (Q, K, V)$
 $\rightarrow$ masked attention $\rightarrow$ MHA $\rightarrow$ FFN
 $\rightarrow$ logits $\rightarrow$ softmax $\rightarrow$ next-token loss.